

CodeShield: ML-Based Code Vulnerability Detection and Risk Assessment Platform

Project Proposal for UCS503

Submitted to:

Ms. Paramveer Kaur

Thapar Institute of Engineering and Technology

Rhea Balamurugan

Roll No.: 1024030152

rbalamurugan_be24@thapar.edu

Ishi Govil

Roll No.: 1024030161

igovil_be24@thapar.edu

Shreya Kush

Roll No.: 1024030718

skush_be24@thapar.edu

1 Higher-order Goal

CodeShield aims to improve software security by automatically identifying potential vulnerabilities in source code before it is deployed. The engineering objective is to build a deployable platform that combines static code analysis with machine learning to detect, classify, and prioritize security vulnerabilities, giving developers a fast and reliable way to catch risky code early in the development cycle.

2 Problem Statement

Modern software projects contain thousands of lines of code, which makes manual security inspection difficult, time-consuming, and error-prone. CodeShield is motivated by the following recurring challenges:

- **Large codebases:** Manually reviewing every line of a real-world project is impractical.
- **Hidden vulnerabilities:** Security issues may not produce visible errors during normal execution, so they go unnoticed until exploited.
- **Late detection:** Vulnerabilities discovered after deployment are significantly more expensive and disruptive to fix.
- **Different vulnerability types:** SQL injection, command injection, hardcoded credentials, and insecure input handling each require different detection strategies.
- **Prioritization:** Developers need to know which of the detected vulnerabilities are the most critical to fix first.

There is therefore a need for an automated system that can analyze source code and provide developers with the vulnerability type, its location, its severity, and actionable security guidance.

3 Proposed Solution

We propose CodeShield, an ML-driven code vulnerability detection platform that accepts source code or a connected repository, analyzes it using a combination of static code analysis and machine-learning techniques, and generates a structured security report.

The main components are:

- **Code submission:** Upload individual source files or connect a Git repository.
- **Code preprocessing:** Parse and normalize source code into an analyzable form.
- **Static analysis:** Identify suspicious coding patterns and program structures using rule-based checks.
- **ML-based detection:** Classify potentially vulnerable code using trained models.
- **Risk assessment:** Assign a severity and confidence score to every detected vulnerability.
- **Security report:** Present the vulnerability type, its exact location, and a plain-language explanation.
- **Dashboard:** Let developers view scan history, trends, and outstanding issues over time.

4 Core Workflow

Figure 1 shows the primary scan pipeline: submitted code passes through preprocessing and structural analysis, is evaluated by rule-based and ML-based engines, and is then merged into a single risk assessment that drives the final report and dashboard.

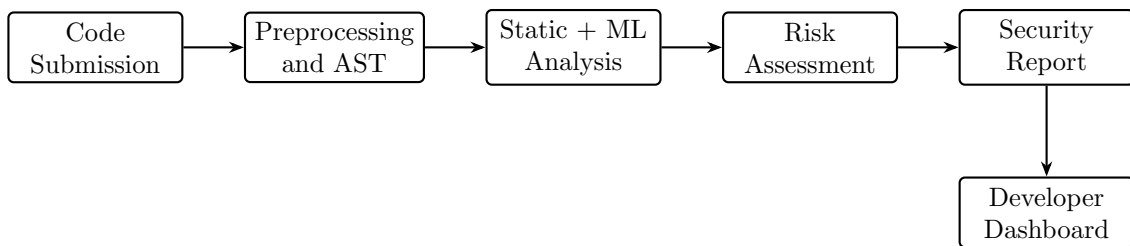


Figure 1: CodeShield core scanning workflow from code submission to the developer-facing dashboard.

4.1 Worked Example

Suppose the submitted code contains the following line:

```
query = "SELECT * FROM users WHERE id=" + user_id
```

CodeShield would report this as:

- **Vulnerability:** SQL Injection – High Severity
- **Location:** database.py, Line 12
- **Reason:** User-controlled input is directly concatenated into an SQL query without sanitization or parameterization.

5 Solution Approach

5.1 Code Preprocessing

Submitted source code is first processed and converted into a representation suitable for analysis. Depending on the programming language, the system uses Abstract Syntax Trees (ASTs) or other code-parsing techniques to understand structures such as:

- Function calls
- Variable declarations and usage
- Control flow
- Imports and dependencies
- User inputs
- Database operations

5.2 Static Analysis

The system identifies known suspicious patterns using predefined security rules, forming a deterministic layer that does not rely entirely on machine learning.

For example, a rule may check whether user-controlled input is directly passed into a system command, which could indicate a potential command injection vulnerability.

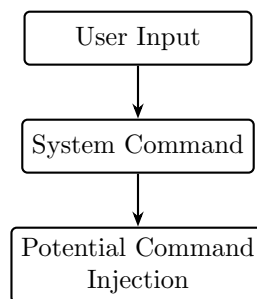


Figure 2: Example of a rule-based static analysis check.

5.3 ML-Based Vulnerability Detection

A machine-learning component analyzes code representations and classifies potentially vulnerable code into a defined set of supported vulnerability categories, rather than claiming to detect every possible vulnerability.

The initial supported categories are:

- SQL Injection
- Command Injection
- Hardcoded Credentials
- Insecure File Handling
- Cross-Site Scripting
- Weak Authentication

Framing the scope as “selected vulnerability categories” keeps the platform’s claims realistic and measurable rather than promising exhaustive coverage.

5.4 Risk Assessment

Every detected issue is assigned:

- Vulnerability type
- Severity
- Confidence score
- Affected file and line number
- Explanation

The platform then prioritizes the most critical vulnerabilities so developers know what to fix first.

6 System Architecture

Figure 2 presents the end-to-end architecture, from the developer-facing frontend through the analysis pipeline to persistent storage and reporting.

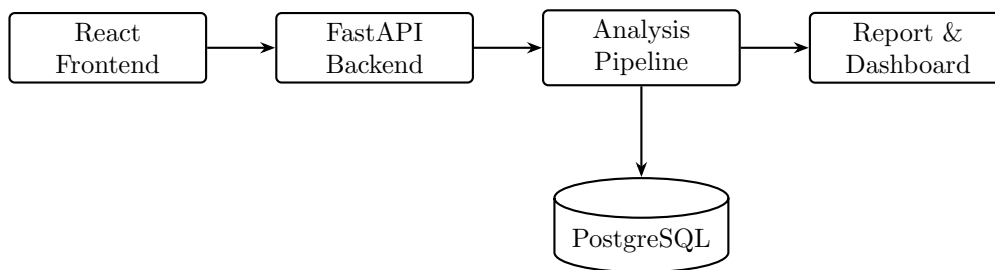


Figure 3: CodeShield system architecture combining a React frontend, FastAPI backend, analysis pipeline, and PostgreSQL database.

7 Technology Stack

Layer	Technologies
Frontend	React.js, HTML/CSS, JavaScript
Backend	Python, FastAPI
Machine Learning	Python, scikit-learn / PyTorch, code embeddings, classification and anomaly-detection techniques
Code Analysis	Python AST; Tree-sitter for multi-language support
Database	PostgreSQL
Optional / Future Extensions	Redis for caching and background processing, Docker for isolated scanning environments, GitHub API for repository scanning, and Celery/RQ for asynchronous analysis

Optional components are presented as possible future extensions rather than commitments for the initial release.

8 Development and CI/CD Pipeline

- **Continuous Integration (CI):** Automatically installs dependencies, runs unit tests, performs code-quality checks, and validates the application whenever new code is pushed to the project repository.

- **Continuous Delivery/Deployment (CD):** After successful checks, the application is built and prepared for deployment to the target environment.
- **Workflow:** Code Push → CI Checks → Tests → Build → CD → Deployment.
- The pipeline ensures that only tested and validated code is integrated and deployed.

9 Evaluation Criteria

CodeShield defines measurable evaluation criteria rather than relying on a general claim of accuracy.

9.1 ML Evaluation

- Precision
- Recall
- F1-score
- Accuracy

Precision and recall are treated as the most important metrics, since both false alarms and missed vulnerabilities carry real cost.

9.2 System Evaluation

- Average code scanning time
- Number of files successfully analyzed
- Detection coverage for supported vulnerability categories
- API response time
- Successful scan completion rate

10 Scalability

The code-analysis pipeline is designed to be separated from the API layer, so multiple scans can be processed without blocking normal API requests.

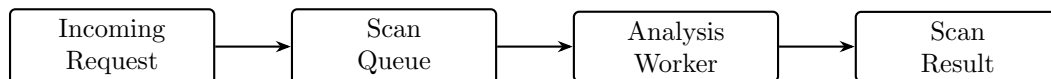


Figure 4: Decoupled scanning pipeline using queued analysis workers.

The system is also designed to grow in language coverage, starting from Python only and extending to Python, JavaScript, Java, and C/C++ using language-specific parsers.

11 Initial Deliverables

The first iteration keeps the scope realistic and measurable:

- User / code submission interface
- FastAPI backend
- Python source-code parser
- Static vulnerability rules
- Initial ML vulnerability classifier
- PostgreSQL database
- Vulnerability report
- Basic security dashboard
- Unit testing and CI/CD pipeline

11.1 Subsequent Deliverables

- GitHub repository scanning
- Support for additional programming languages
- Improved ML model
- Severity prioritization
- Code-level remediation suggestions
- Scan history and comparison
- Background / asynchronous scanning
- Docker-based isolated analysis

12 Risks and Mitigation

Risk	Mitigation
False positives – the system may flag safe code as vulnerable	Combine ML predictions with static-analysis rules and report a confidence score rather than automatically declaring code unsafe.
False negatives – some vulnerabilities may be missed	Clearly define supported vulnerability categories and evaluate performance using recall.
Dataset limitations – labelled vulnerable code can be hard to obtain	Start with a limited set of vulnerability classes and use validated datasets and examples.
Programming language differences – security patterns vary by language	Initially support one or two languages rather than claiming universal coverage.
Security of uploaded code – source code may contain confidential information	Minimize storage, restrict access, and process uploaded code in isolated environments.

13 Summary

CodeShield is a software-security platform that combines static code analysis and machine learning to automatically identify selected vulnerabilities in source code. The system analyzes submitted code, classifies potential security issues, estimates their severity, and presents developers with an actionable, prioritized security report through a web-based dashboard.

The project also incorporates a CI/CD pipeline to automate testing, validation, and deployment of the platform, helping the development team maintain a stable and reliable application.